

PROMPT ADVISERS / GPT-6 ASTRA

COMPUTER USE

FIELD GUIDE

Five workflows. Better prompts. A reusable flight-search CLI.



COPY. CUSTOMIZE. RUN. VERIFY.

Pick one real job.

The prompts work best when the app, inputs and finish line are explicit. Start with one bounded task, then expand after you can verify its result.

Your five starting points

- 01 Turn searches into a tool / page 3
- 02 Test tools inside their app / page 4
- 03 Let one agent test another / page 5
- 04 Finish the editing handoff / page 6
- 05 Use your phone from your Mac / page 7

Before the first run

Use a Codex task that actually exposes the computer-use tools your task needs. Sign in to the target apps yourself. A model name alone does not guarantee that a browser, desktop or phone-control tool is enabled.

Replace every <PLACEHOLDER> with your own app, file, route or account. Use a test copy for changes. For secrets or sign-in challenges, finish that step yourself and then let the task continue.

The four-part prompt contract

APP Name the exact app, workspace or browser.

RESULT Describe the useful thing you want at the end.

SCOPE State which files or settings may change.

PROOF Say how the agent should verify completion.

Everything you need to copy

The ZIP includes one text file per prompt, ALL-PROMPTS.md and the CLI source. Use the text files for reliable copying. The PDF explains setup, evidence and adaptation.

These are practical templates, not guaranteed outcomes. Interface changes, account access and missing tools can block a run. The useful response is an exact blocker with evidence.

Turn searches into a tool

Build a reusable path from a browser search to checked JSON and CSV.

Use Codex desktop with its supported computer-use browser. For the included CLI, install Node 22+, choose future dates, and use business class in CAD. Start with the existing CLI on page 8 if you want the fastest path.

Copy, then fill your details

Build a reusable browser-backed flight-search CLI for this request:

Origin: <ORIGIN_AIRPORT>
 Destination: <DESTINATION_AIRPORT>
 Departure / return: <YYYY-MM-DD> / <YYYY-MM-DD>
 Adults: <COUNT>
 Cabin / currency: business / CAD
 Output folder: <PROJECT_FOLDER>

First inspect Google Flights using Codex's supported internal browser. Read the active tool documentation and observe the actual controls. Separate the local planner/parser from the browser adapter, and state exactly which parts require Codex. Do not claim a shell command can retrieve live fares if it cannot.

Create explicit inputs for the route and dates. Export airline, displayed total for the whole party, outbound stops and duration, warnings, source link, price scope and retrieval time as JSON and CSV. Keep unverified return details and all-segment cabin checks marked unknown. Missing data must not become zero.

Run two future date pairs in fresh output folders. Verify the visible airports, full dates including year, adults, cabin and currency. Wait for settled fare cards and compare one exported result with the page. Preserve the capture and original timestamp.

Deliver a quickstart, browser dependency notes, synthetic parser tests and the two verification reports. Search only; do not book or enter traveler or payment information.

Completion proof Two distinct date pairs produce separate runs. At least one exported fare matches the visible result. The report distinguishes a displayed from-price from a selected complete itinerary.

Make it yours Apply the same pattern to another repeated search: specify inputs, inspect the interface once, save a supported route, and verify every result.

Test tools inside their app

Check the whole user journey, including tool invocation, follow-ups and source accuracy.

Have the host app installed and signed in. Supply your own MCP or integration name and a safe source. If installation is part of the test, provide its documented install command and the exact host app.

Copy, then fill your details

Use computer use to test my existing integration inside its actual host app.

Host app and workspace: <APP_AND_WORKSPACE>

Tool / MCP / integration: <TOOL_NAME>

Safe test source or topic: <SOURCE_OR_TOPIC>

Expected successful result: <EXPECTED_RESULT>

Optional source checkout: <REPO_PATH_OR_NONE>

Confirm you are in the right app and workspace. Check whether the tool is available. If setup is required, use the supplied installation instructions, preserve existing configuration, and report exactly what changed. Stop at any login or secret entry that needs me. Do not rebuild a tool that already exists.

Run a realistic read-only request. Inspect the actual tool-call details, not just the final assistant message. Ask two natural follow-ups that depend on the first answer. Include one missing-source or no-result case to check whether it handles uncertainty honestly.

Verify one material answer against the underlying source. Record the observed inputs, output, errors and elapsed time. Separate model response time, tool execution time and any setup friction when evidence supports that distinction. Do not invent timing breakdowns.

Return a short report: passed checks, reproduced failures, evidence, and the highest-value fix. If source code is available, identify the likely location without applying changes in this test. Keep private records out of the report. Do not send messages to other people.

Completion proof The host app invokes the intended tool; follow-ups retain context; a returned claim matches its source; the no-result case does not invent an answer.

Make it yours Use this for an MCP, API-backed feature, skill or plugin. Replace the host app and success condition; preserve the same evidence checks.

Let one agent test another

Reproduce an app failure, connect it to source code, and retest the same user journey.

Provide the exact app and matching source checkout. Use an isolated development environment and a clean test conversation. Repair mode is optional; the default below authorizes up to three local repair attempts.

Copy, then fill your details

Test my app through its real interface, including its own computer-use feature.

App: <APP_NAME_OR_PATH>

Matching source repository: <REPO_PATH>

Browser task: <SMALL_READ_ONLY_TASK>

Source to verify against: <PUBLIC_SOURCE_URL>

Success criteria: <OBSERVABLE_PASS_CONDITIONS>

Open a clean test conversation in the correct app. Ask the app's agent to perform the browser task using its own computer-use capability. Observe its actions and tool output. Do not substitute your own browsing for the app's work and then mark the app as passing.

Check one returned fact against the public source. Close and reopen the conversation to test persistence. Capture the precise point of failure if the task stalls, loses context, uses the wrong window or returns an unsupported claim.

If it fails, reproduce the failure and verify that the supplied checkout builds this app. In the local development copy, fix the smallest cause, run relevant tests, rebuild or relaunch as needed, and repeat the identical interface test. Preserve unrelated work. Limit this run to three repair attempts; if still failing, return the evidence and next diagnostic step.

Return before/after behavior, changed files, test results, any remaining failure, and a reproducible request. Do not deploy publicly, access private conversations, make purchases or send external messages.

Completion proof The tested app performs its own actions. A failure is recorded rather than hidden. Any repair passes the same test, and the conversation survives reopening.

Make it yours For an app without an agent, replace the browser task with a concrete user journey such as creating a draft project, saving it, and reopening it.

Finish the editing handoff

Move from a finished render to a conservatively cleaned, checked export.

Have Descript installed and authenticated, and supply a short test video first. The cleanup prompt is included on pages 10–11 and in its own text file; no private custom skill is required.

Copy, then fill your details

Run my post-render editing workflow using the completed sample below.

Source video: <SOURCE_VIDEO_PATH>

Output folder: <OUTPUT_FOLDER>

Output filename: <NEW_FILENAME>

Optional Drive folder: <FOLDER_URL_OR_NONE>

Cleanup prompt file: <PATH_TO_07_DESCRIPT_CLEANUP_TXT>

Verify that the source file is stable and decodable; record its duration, resolution and audio properties. Preserve the original. Create a fresh Descript project and import the video. Use the app's available CLI or API for supported steps and computer use for the interface steps that require it. Wait for import and transcription to finish.

Submit the complete supplied conservative-cleanup prompt. Wait for the edit to finish, then compare the original and edited transcripts. Keep the final retake and every unique point. Watch and listen around changed cuts, the opening and the ending. Repair clipped words, missing unique content or awkward joins before exporting.

Export to the new filename at the source resolution. Verify that the exported file decodes, contains picture and sound, and matches the intended edit and resolution. Report source versus export duration without treating a shorter edit as proof of quality.

If I supplied a Drive folder, upload the verified export and check its filename, size and destination. Otherwise save locally. Return the local path, optional Drive link and verification summary. Do not send it to anyone or publish publicly. Stop with the exact blocker if authentication or a required control is unavailable.

Completion proof The original remains untouched. The complete cleanup prompt was submitted. Unique content survives, cut boundaries sound natural, and the actual exported media is checked.

Make it yours For an unfinished render, explicitly request a scheduled follow-up supported by your environment. Watch file stability and decodability; a screenshot or a filename alone does not prove completion.

Use your phone from your Mac

Apply a specific mobile-app setting and prove it stayed changed.

Open and authenticate iPhone Mirroring yourself first. Use compatible devices and Apple's current setup requirements. Choose the correct account and explicit settings; do not promise that a setting improves reach.

Copy, then fill your details

Use the iPhone Mirroring window I opened to configure Instagram on my actual phone.

Account to confirm: <ACCOUNT_HANDLE>

Requested change: enable Upload at highest quality, if that control is available

Other approved changes: <SETTING_AND_TARGET_VALUE_OR_NONE>

Confirm the visible account before changing anything. Inspect the current settings and explain the effect of each requested change. Use the labels actually present in this app version; do not assume a remembered menu path is still correct.

Apply only the requested changes. If a requested value is already set, record it as already correct. Leave the settings screen, reopen it and verify that each value persisted. Keep a before/after table with the visible setting label, old value, new value and verification result.

If you find other potentially useful creator settings, list them separately as suggestions. Distinguish upload quality from claims about distribution or reach. Verify any platform-policy or performance claim against current official documentation before recommending it.

Do not alter account privacy, security, recovery details, profile identity or paid features. Do not publish, send DMs or browse unrelated photos. If mirroring, login or an unavailable setting blocks the task, stop at that screen and tell me the exact action needed. Return completed, already-correct and blocked items separately.

Completion proof The correct account is visible, the named setting reaches the target value, and the value persists after reopening the screen.

Make it yours Use this for another mobile-only app by naming its exact account, setting and target value. Keep the before/after/persistence check.

Skip straight to reuse.

The public repository contains the same reusable search approach, cleaned for public use. No private travel data or account setup is included.

github.com/promptadvisers/google-flights-cli

1 / Try the offline demo

```
git clone https://github.com/promptadvisers/google-flights-cli.git
cd google-flights-cli
npm run demo
```

Node 22+ is enough. No npm install or API key is required. The demo produces synthetic JSON and CSV, clearly labeled as invented test data.

2 / Make your own request

```
npm run example
```

Edit the generated request: your airports, future dates and adult count. Current live support: business class, CAD, 1-9 adults. This version does not support economy, USD, children, infants or award-seat searches.

3 / Let Codex run the browser stage

```
Read README.md and docs/BROWSER-RUNBOOK.md in <REPO_PATH>.
Run flight-search.mjs run --request <REQUEST_JSON>
with a fresh output folder. Execute its returned browser steps
through the supported internal browser. Verify the settings
and settled cards, then return the JSON, CSV and retrieval time.
```

How the pieces connect

Your request > local plan > Codex browser

Google Flights > verified capture > JSON / CSV

The command plans the work; the browser fetches fresh results. Bare Node, cron or another agent without a compatible browser cannot execute this live adapter. Price-drop checks compare fresh captures with prior history; they do not fetch new prices by themselves.

Explore. Then measure.

Use these after a basic search succeeds. Avoid confusing a larger search plan with a completed search.

Explore routes with a budget

Extend the flight search using <BASE_REQUEST> and these explicit alternatives: <DATE_WINDOW>, <ALTERNATE_AIRPORTS>, <OPEN_JAW_OR_STOPOVER_OPTIONS>. Use the included campaign planner with a maximum of <SEARCH_BUDGET> browser searches. Explain which combinations are omitted.

Start from the original route. Use the visible date-grid and airport controls to narrow promising options, then capture exact requests. Include ground transport, positioning flights, bags and extra hotel nights in comparisons; leave unknown costs unknown. Treat separate-ticket trips as requiring connection and ticket-protection review. Do not claim that a longer connection is feasible merely because the dates are chronological.

Rank only complete, fresh observations with matching price scope. Keep planning-only strategies and incomplete component pairs separate. Do not claim the global cheapest fare or invent award-seat availability. Return a shortlist with evidence links, retrieval times, missing costs and what still needs checking.

Check whether reuse helps

Compare the reusable flight runner with direct computer use on <TWO_NEW_ROUTE_AND_DATE_REQUESTS>. Use separate fresh task contexts with the same browser capability, model settings, route, dates, adults, cabin, currency and result coverage. Alternate which method runs first.

Define the timer start and finish before testing. Record elapsed time, tool calls, failed attempts and final evidence. Compare all exported fares and warnings against equivalent visible results. If prices change between runs, record the mismatch instead of treating it as a method error.

Report each run and the small-sample limits. Distinguish navigation-to-export time from total task time. Do not claim a universal speedup from two searches or call a faster incomplete result a success.

Keep the final take.

Paste the full 07-descript-cleanup.txt file into the editor. These two pages show the complete instructions.

Clean up this tutorial video segment. Treat the supplied video as a standalone segment and apply the rules below. If it is part of a longer recording, use the pacing appropriate to its role.

GOAL

Remove retakes, filler words, and tighten extended pauses while keeping the delivery natural. Conservative editing - when uncertain, leave it in.

NON-NEGOTIABLE RULES (apply these first):

1. Always keep the LAST/final version of any retake. Cut the earlier failed attempts, never the final clean one.
2. Never cut unique content. Only cut clear retakes and abandoned attempts. It is not your job to judge whether content is "worth keeping" - if a sentence says something new, it stays.
3. Leave buffer around every cut. Do not splice so tightly that the last word of the kept take gets clipped or the next clip starts before the previous word fully finishes. Err toward a few extra frames of breathing room over an abrupt jump.

WHAT COUNTS AS A RETAKE

I frequently stutter, repeat similar sentences, or abandon a sentence mid-way and restart with different phrasing. All of these are retakes:

- Repeated openings (starting the same sentence two or more times).
- Mid-sentence restarts and self-corrections.
- Filler stutters ("but the, I...", "so it's, it's like...").
- Trailing off, stammering, or visibly giving up on a sentence, then pivoting to say the same idea a cleaner way.
- A sentence that cuts off and is immediately followed by a rephrased version of the same point.

For all of the above: cut the failed attempt(s), keep the clean final version.

An editing assistant reporting success is not verification. Compare the transcript, listen to cut boundaries and inspect the actual exported video before handing it off.

Keep the meaning.

Paste the full 07-descript-cleanup.txt file into the editor. These two pages show the complete instructions.

SEPARATE RETAKES FROM UNIQUE CONTENT

A messy block often mixes failed attempts AND unique sentences. Do not delete the whole block.

- If a block has 3 failed starts followed by a clean unique sentence, cut only the 3 failed starts and keep the unique sentence.
- Any sentence that introduces new or different information stays, even when surrounded by retakes.

COHERENCE CHECK (after cutting)

- The result must flow naturally - no orphaned transition words, no jarring jumps, no half-pronounced words bleeding into the next clip.
- Re-listen to each cut point and confirm the audio lands cleanly. If a cut sounds abrupt, restore a little buffer rather than leaving it tight.

PACING

- If this segment is the intro/hook (the very beginning of the full video), keep it tight - minimize spacing between sentences for energy.
- For all other segments (body and outro), use natural pacing with more nuance - don't over-compress.
- Tighten extended pauses and dead gaps everywhere, but keep enough breathing room that it sounds human, not robotic or rushed.

An editing assistant reporting success is not verification. Compare the transcript, listen to cut boundaries and inspect the actual exported video before handing it off.

Make the pattern yours.

Start from a job you already do. A precise finish line is more useful than a long list of features.

Use <EXACT_APP_OR_BROWSER> to complete <OBSERVABLE_OUTCOME>.

Inputs: <FILES, URLs, ACCOUNT_OR_WORKSPACE>

Scope: <WHAT_MAY_CHANGE>

Success: <VISIBLE_ACCEPTANCE_CHECKS>

Deliverables: <FILES_AND_REPORT>

Budget: <TIME_OR_ATTEMPT_LIMIT>

Inspect the current state and active tool documentation before acting. Use the intended app's own capabilities where that is what we are testing. Keep source material and unrelated work intact. Make reasonable choices inside the requested scope; ask only for missing information that materially changes the result.

After the work, verify the result in the interface or exported artifact. If it fails, reproduce the problem and return the evidence. Report what completed, what remains unverified, and the exact next step.

When a run goes wrong

Wrong app or account Stop the action and confirm the visible target.

Missing tool State the required capability and the available fallback.

Changed interface Inspect the current screen instead of repeating old coordinates.

Unexpected result Compare it with the source; keep the failure evidence.

Partial success List what completed and what still needs a check.

Keep the evidence.

The resource gives you prompts and source code. Your environment supplies the apps, accounts and enabled computer-use tools.

CLI source and documentation

Start here for current supported inputs, commands, source code and issue reporting.

<https://github.com/promptadvisers/google-flights-cli>

iPhone Mirroring setup

Check Apple's current compatibility and setup requirements before trying the phone workflow.

<https://support.apple.com/en-us/120421>

Codex documentation

Use current official setup documentation and the active tool descriptions for your environment.

<https://developers.openai.com/codex/>

A small handoff that is actually useful

For each job, keep the request, relevant before/after evidence, output files, retrieval or execution time, and a short note explaining what remains unverified. Remove private account data before sharing examples.

About this edition

Prepared September 5, 2026 for the GPT-6 Astra computer-use video. Prompts are adapted from the five demonstrated workflows, with clearer inputs, limits and completion checks. The original app-agent demo found a failure; this guide does not claim every example succeeded without intervention. The video-editing demo used an already-rendered sample.

Prompt Advisers

Independent educational material. Google, OpenAI, Apple, Instagram and Descript are referenced to identify the tools in the workflows. This is not an official product of those companies.